LAPORAN PRAKTIKUM

Struktur Data

“Single linkedList”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan penulisan makalah yang berjudul *Single Linked List* ini dengan baik.

Makalah ini disusun dengan tujuan untuk memenuhi tugas serta menambah pemahaman mengenai struktur data, khususnya konsep *Single Linked List*. Dalam dunia pemrograman, pemahaman terhadap struktur data sangatlah penting karena berperan dalam pengolahan dan pengelolaan data secara efisien. *Single Linked List* merupakan salah satu struktur data dinamis yang banyak digunakan karena fleksibilitasnya dalam penambahan dan penghapusan data.

Penulis menyadari bahwa dalam penyusunan makalah ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca agar dapat memperbaiki karya di masa yang akan datang.

Akhir kata, penulis berharap makalah ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca, khususnya dalam memahami konsep *Single Linked List* dalam struktur data.

Abdul Jabbar

Padang, 5 Mei 2026

DAFTAR ISI

BAB I 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 2
2.1 Pengertian 2
2.2 Operasi Operasi yang di gunakan dalam single LinkedList 3
2.3 Praktikum pekan 5 4
2.3.1 class NodeSLL_2511533021 4
2.3.2 class PencarianSLL_2511533021 5
2.3.3 class TambahSLL_2511533021 7
2.3.4 class HapusSLL_2511533021 8
BAB III KESIMPULAN 12
3.1 Kesimpulan 12
3.2 Saran 12
DAFTAR PUSTAKA 13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan dunia teknologi informasi, kebutuhan akan pengolahan data yang efisien semakin meningkat. Struktur data menjadi salah satu komponen penting dalam pemrograman karena menentukan bagaimana data disimpan, diakses, dan dikelola. Salah satu struktur data dasar yang sering digunakan adalah *Single Linked List*.

Single Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node menyimpan data dan referensi (pointer) ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, *Single Linked List* bersifat dinamis sehingga memungkinkan penambahan dan penghapusan elemen dengan lebih fleksibel.

Pemahaman mengenai *Single Linked List* sangat penting bagi mahasiswa informatika karena menjadi dasar dalam mempelajari struktur data yang lebih kompleks. Selain itu, konsep ini juga banyak diterapkan dalam berbagai aplikasi, seperti manajemen memori, implementasi stack dan queue, serta sistem pengolahan data lainnya.

1.2 Tujuan

1. Memahami konsep dasar *Single Linked List* dalam struktur data.
2. Mengetahui cara kerja *Single Linked List* dalam menyimpan dan mengelola data.
3. Mempelajari operasi dasar pada *Single Linked List*, seperti penambahan, penghapusan, dan traversal data.
4. Meningkatkan pemahaman dalam penerapan struktur data pada pemrograman.

1.3 Manfaat

1. Menambah wawasan pembaca mengenai struktur data, khususnya *Single Linked List*.
2. Membantu mahasiswa dalam memahami konsep dasar pemrograman yang lebih kompleks.
3. Menjadi referensi dalam pembelajaran dan praktik struktur data.
4. Meningkatkan kemampuan dalam merancang program yang efisien dan terstruktur.

BAB II
PEMBAHASAN

2.1 Pengertian

Linked list adalahsekumpulan elemen bertipe sama, yang mempunyai keterurutan tertentu, yang setiap elemennya terdiri dari dua bagian. Struktur berupa rangkaian elemen saling berkait dimana setiap elemen dihubungkan elemen lain melalui pointer. Pointer adalah alamat elemen.Penggunaan pointer untuk mengacu elemen berakibat elemen-elemen bersebelahan secara logik walau tidak bersebelahan secara fisik di memori.

Terdapat tempat yang disediakan pada satu area memori tertentu untuk menyimpan data dikenal dengan sebutan node atau simpul.Setiap node memiliki pointer yang menunjuk ke simpul berikutnya sehingga terbentuk satu untaian, dengan demikian hanya diperlukan sebuah variabel pointer.Susunan berupa untaian semacam ini disebut Single Linked List (NULL memilik nilai khusus yang artinya tidak menunjuk ke mana-mana. Biasanya Linked List pada titik akhirnya akan menunjuk ke NULL). Salah satu kelemahan single linked list adalah pointer (penunjuk) hanya dapat bergerak satu arah saja, maju/mundur, atau kanan/kiri sehingga pencarian data pada single linked list hanya dapat bergerak dalam satu arah saja. Untuk mengatasi kelemahan tersebut, dapat menggunakan metode double linked

list. Linked list ini dikenal dengan nama Linked list berpointer Ganda atau Double Linked List.

Pembuatan Single Linked List dapat menggunakan 2 metode: LIFO (Last In First Out), aplikasinya : Stack (Tumpukan) FIFO (First In First Out), aplikasinya : Queue (Antrean).

2.2 Operasi Operasi yang di gunakan dalam single LinkedList

1. *Insert*

   Istilah Insert berarti menambahkan sebuah simpul baru ke dalam suatu linked list.

2. *IsEmpty*

   Fungsi ini menentukan apakah linked list kosong atau tidak.

3. *FindFirst*

   Fungsi ini mencari elemen pertama dari linked list.

4. *FindNext*

   Fungsi ini mencari elemen sesudah elemen yang ditunjuk now.

5. *Retrieve*

   Fungsi ini mengambil elemen yang ditunjuk oleh now. Elemen tersebut lalu dikembalikan oleh fungsi.

6. *Update*

   Fungsi ini mengubah elemen yang ditunjuk oleh now dengan isi tertentu.

7. *DeleteNow*

   Fungsi ini menghapus elemen yang ditunjuk oleh now. Jika yang dihapus adalah elemen pertama dari linked list (head), maka head akan berpindah ke elemen berikutnya.

8. *DeleteHead*

   Fungsi ini menghapus elemen yang ditunjuk head. Head akan berpindah ke elemen sesudahnya.

9. *Clear*

   Fungsi ini menghapus seluruh linked list yang ada. Fungsi ini wajib dilakukan saat mengakhiri program yang menggunakan linked list agar data yang dialokasikan di memori tidak tertinggal.

2.3 Praktikum pekan 5

Pada pekan 5 mahasiswa informatika unand mempelajari kode program mengenai single LinkedList

2.3.1 class NodeSLL_2511533021

```java
package pekan5_2511533021;

public class NodeSLL_2511533021 {
    // node bagian data
    int data_3021;

    // Pointer ke node berikutnya
    NodeSLL_2511533021 next_3021;

    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533021(int data_3021) {
        this.data_3021 = data_3021;
        this.next_3021 = null;
    }
}
```

1. int data_3021;Variabel ini digunakan untuk menyimpan data dalam node dengan tipe integer.
2. NodeSLL_2511533021 next_3021;Variabel ini adalah pointer (referensi) yang menunjuk ke node berikutnya dalam linked list.
3. public NodeSLL_2511533021(int data_3021) {

   Konstruktor yang digunakan untuk menginisialisasi node saat pertama kali dibuat.
4. this.data_3021 = data_3021;Mengisi nilai data pada node sesuai dengan parameter yang diberikan.
5. this.next_3021 = null;Mengatur pointer next menjadi null, artinya node belum terhubung ke node lain.

2.3.2 class PencarianSLL_2511533021

```java
package pekan5_2511533021;

public class PencarianSLL_2511533021 {

    static boolean searchKey(NodeSLL_2511533021 head_3021, int key_3021) {
        NodeSLL_2511533021 curr_3021 = head_3021;

        while (curr_3021 != null) {
            if (curr_3021.data_3021 == key_3021)
                return true;

            curr_3021 = curr_3021.next_3021;
        }

        return false;
    }

    public static void traversal(NodeSLL_2511533021 head_3021) {
        NodeSLL_2511533021 curr_3021 = head_3021;

        while (curr_3021 != null) {
            System.out.print(" " + curr_3021.data_3021);
            curr_3021 = curr_3021.next_3021;
        }

        System.out.println();
    }

    public static void main(String[] args) {

        NodeSLL_2511533021 head_3021 = new NodeSLL_2511533021(14);
```

```java
        NodeSLL_2511533021 head_3021 = new NodeSLL_2511533021(14);
        head_3021.next_3021 = new NodeSLL_2511533021(21);
        head_3021.next_3021.next_3021 = new NodeSLL_2511533021(13);
        head_3021.next_3021.next_3021.next_3021 = new NodeSLL_2511533021(30);
        head_3021.next_3021.next_3021.next_3021.next_3021 = new NodeSLL_2511533021(10);

        System.out.print("Penelusuran SLL : ");
        traversal(head_3021);

        int key_3021 = 30;
        System.out.print("cari data " + key_3021 + " = ");

        if (searchKey(head_3021, key_3021))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

1. *Method searchKey*

   Method ini digunakan untuk mencari suatu nilai (key) dalam linked list. Parameter:

   -head_3021 sebagai node awal

   - key_3021 sebagai data yang dicari

2. *Proses Pencarian*

   - Menggunakan perulangan while selama node tidak null.
   - Setiap node dicek apakah datanya sama dengan key.
   - Jika sama, method mengembalikan nilai true.
   - Jika tidak, lanjut ke node berikutnya.

3. *Return False*

   ka data tidak ditemukan sampai akhir linked list, method mengembalikan falsJie.

4. *Method traversal*

   Digunakan untuk menampilkan seluruh isi linked list. Menelusuri node dari head sampai null dan mencetak setiap data.

5. *Method main*

   Merupakan titik awal eksekusi program.

6. Pembuatan Linked ListMembuat node pertama sebagai head dengan nilai 14.

   Node berikutnya dihubungkan menggunakan pointer next.Data yang dimasukkan adalah 14, 21, 13, 30, dan 10.

7. Menampilkan *Linked List*

   Method traversal dipanggil untuk menampilkan seluruh isi linked list.

8. *Penentuan Key*

   Variabel key_3021 diisi dengan nilai yang akan dicari, yaitu 30.

9. *Proses Pencarian Data*

   Method searchKey dipanggil untuk mencari data dalam linked list.

10. Menampilkan Hasil Jika data ditemukan, ditampilkan "ketemu".Jika tidak ditemukan, ditampilkan "tidak ada"

2.3.3 class TambahSLL_2511533021

```java
package pekan5_2511533021;

public class TambahSLL_2511533021 {
    static NodeSLL_2511533021 insertPos(NodeSLL_2511533021 headNode_3021, int position_3021, int value_3021) {
        NodeSLL_2511533021 head_3021 = headNode_3021;

        if (position_3021 < 1)
            System.out.print("Invalid position");

        if (position_3021 == 1) {
            NodeSLL_2511533021 new_node_3021 = new NodeSLL_2511533021(value_3021);
            new_node_3021.next_3021 = head_3021;
            return new_node_3021;
        } else {
            while (position_3021-- != 0) {

                if (position_3021 == 1) {
                    NodeSLL_2511533021 newNode_3021 = GetNode_3021(value_3021);
                    newNode_3021.next_3021 = headNode_3021.next_3021;
                    headNode_3021.next_3021 = newNode_3021;
                    break;
                }

                headNode_3021 = headNode_3021.next_3021;
            }

            if (position_3021 != 1)
                System.out.print("Posisi di luar jangkauan");
        }

        return head_3021;
    }

    public static void printList(NodeSLL_2511533021 head_3021) {
        NodeSLL_2511533021 curr_3021 = head_3021;

        while (curr_3021.next_3021 != null) {
            System.out.print(curr_3021.data_3021 + "-->");
            curr_3021 = curr_3021.next_3021;
        }

        if (curr_3021.next_3021 == null) {
            System.out.print(curr_3021.data_3021);
        }

        System.out.println();
    }

    public static void main(String[] args) {

        // tambah di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_3021 = 7;
        head_3021 = insertAtEnd(head_3021, data2_3021);
        printList(head_3021);

        // tambah di posisi tertentu
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_3021 = 4;
        int pos_3021 = 4;
        head_3021 = insertPos(head_3021, pos_3021, data3_3021);
        printList(head_3021);
    }
}
```

1. *Method insertAtFront*

Digunakan untuk menambahkan node di bagian depan linked list.Nodebaru dibuat lalu diarahkan ke head lama, kemudian menjadi head baru.

2. *Method insertAtEnd*

Digunakan untuk menambahkan node di bagian akhir linked list.Jika list kosong, node baru langsung menjadi head.Jika tidak, program menelusuri hingga node terakhir lalu menambahkan node baru di akhir.

3. *Method GetNode_*3021

   Digunakan untuk membuat node baru dengan nilai tertentu.

4. *Method insertPos*

   Digunakan untuk menambahkan node pada posisi tertentu.Jika posisi 1, maka sama seperti insert di depan. Jika posisi lainnya, program menelusuri node sampai posisi yang diinginkan lalu menyisipkan node baru. Jika posisi tidak valid, akan menampilkan pesan kesalahan.

   Method printList

   Digunakan untuk menampilkan seluruh isi linked list.Data ditampilkan dengan format panah (-->).

   Method main Merupakan titik awal eksekusi program.

5. Pembuatan Linked List Awal, Membuat linked list dengan data 2, 3, 5, dan 6.
6. Menampilkan Data Awal, Menampilkan isi linked list sebelum dilakukan penambahan.
7. Penambahan di Depan, Menambahkan data 1 ke bagian depan linked list.
8. Penambahan di Belakang, Menambahkan data 7 ke bagian akhir linked list.
9. Penambahan di Posisi Tertentu, Menambahkan data 4 pada posisi ke-4 dalam linked list.

2.3.4 class HapusSLL_2511533021

```java
package pekan5_2511533021;

public class HapusSLL_2511533021 {

    // fungsi untuk menghapus head
    public static NodeSLL_2511533021 deleteHead(NodeSLL_2511533021 head_3021) {
        if (head_3021 == null)
            return null;

        head_3021 = head_3021.next_3021;
        return head_3021;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533021 removeLastNode(NodeSLL_2511533021 head_3021) {
        if (head_3021 == null) {
            return null;
        }

        if (head_3021.next_3021 == null) {
            return null;
        }

        NodeSLL_2511533021 secondLast_3021 = head_3021;

        while (secondLast_3021.next_3021.next_3021 != null) {
            secondLast_3021 = secondLast_3021.next_3021;
        }

        secondLast_3021.next_3021 = null;
        return head_3021;
```

```java
    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533021 deleteNode(NodeSLL_2511533021 head_3021, int position_3021) {
        NodeSLL_2511533021 temp_3021 = head_3021;
        NodeSLL_2511533021 prev_3021 = null;

        if (temp_3021 == null)
            return head_3021;

        if (position_3021 == 1) {
            head_3021 = temp_3021.next_3021;
            return head_3021;
        }

        for (int i = 1; temp_3021 != null && i < position_3021; i++) {
            prev_3021 = temp_3021;
            temp_3021 = temp_3021.next_3021;
        }

        if (temp_3021 != null) {
            prev_3021.next_3021 = temp_3021.next_3021;
        } else {
            System.out.println("Data tidak ada");
        }

        return head_3021;
    }

    // fungsi mencetak SLL
    public static void printList(NodeSLL_2511533021 head_3021) {
        NodeSLL_2511533021 curr_3021 = head_3021;

```

```java

    public static void main(String[] args) {

        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6
        NodeSLL_2511533021 head_3021 = new NodeSLL_2511533021(1);
        head_3021.next_3021 = new NodeSLL_2511533021(2);
        head_3021.next_3021.next_3021 = new NodeSLL_2511533021(3);
        head_3021.next_3021.next_3021.next_3021 = new NodeSLL_2511533021(4);
        head_3021.next_3021.next_3021.next_3021.next_3021 = new NodeSLL_2511533021(5);
        head_3021.next_3021.next_3021.next_3021.next_3021.next_3021 = new NodeSLL_2511533021(6);

        System.out.println("List awal:");
        printList(head_3021);

        // hapus head
        head_3021 = deleteHead(head_3021);
        System.out.println("List setelah head dihapus:");
        printList(head_3021);

        // hapus node terakhir
        head_3021 = removeLastNode(head_3021);
        System.out.println("List setelah simpul terakhir dihapus:");
        printList(head_3021);

        // hapus posisi ke-2
        int position_3021 = 2;
        head_3021 = deleteNode(head_3021, position_3021);
        System.out.println("List setelah posisi 2 dihapus:");
        printList(head_3021);
    }
}
```

1. Method deleteHead

   Digunakan untuk menghapus node pertama (head). Jika list kosong, mengembalikan null. Jika tidak, head dipindahkan ke node berikutnya.

2. Method removeLastNode

   Digunakan untuk menghapus node terakhir.Jika list kosong atau hanya satu node, mengembalikan null. Jika lebih dari satu node, program mencari node sebelum terakhir lalu memutus link ke node terakhir.

3. Method deleteNode

   Digunakan untuk menghapus node pada posisi tertentu. Jika posisi 1, maka head dihapus. Jika posisi lain, program menelusuri hingga posisi yang diinginkan lalu menghapus node tersebut. Jika posisi tidak ditemukan, menampilkan pesan "Data tidak ada".

4. Method printList

   - Digunakan untuk menampilkan seluruh isi linked list.

- Data ditampilkan dengan format panah (-->) antar node.

5. Method main, Merupakan titik awal eksekusi program.

6. Pembuatan Linked List Awal
   - Membuat linked list dengan data 1, 2, 3, 4, 5, dan 6.
   - Menampilkan List Awal
   - Menampilkan isi linked list sebelum dilakukan penghapusan.

7. Penghapusan Head, Menghapus node pertama dari linked list.

8. Penghapusan Node Terakhir,Menghapus node paling akhir dari link ed list.

9. Penghapusan Berdasarkan Posisi, Menghapus node pada posisi ke-2 dalam linked list.

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat disimpulkan bahwa *Single Linked List* merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung melalui pointer. Setiap node memiliki dua komponen utama, yaitu data dan referensi ke node berikutnya.

Dibandingkan dengan array, *Single Linked List* memiliki keunggulan dalam hal fleksibilitas karena ukurannya dapat berubah secara dinamis sesuai kebutuhan. Selain itu, proses penambahan dan penghapusan data dapat dilakukan dengan lebih efisien tanpa harus menggeser elemen lainnya. Namun, struktur ini juga memiliki kelemahan, seperti akses data yang tidak langsung (harus ditelusuri dari awal) serta penggunaan memori tambahan untuk pointer.

Dengan memahami konsep dan operasi dasar seperti *insert*, *delete*, dan *traversal*, mahasiswa dapat lebih mudah mengembangkan program yang efisien serta memahami struktur data yang lebih kompleks.

3.2 Saran

1. Mahasiswa disarankan untuk lebih banyak berlatih dalam mengimplementasikan *Single Linked List* menggunakan berbagai bahasa pemrograman agar pemahaman semakin mendalam.
2. Dalam pembuatan program, sebaiknya memperhatikan pengelolaan memori agar tidak terjadi kebocoran memori (*memory leak*).
3. Perlu dilakukan pengembangan lebih lanjut dengan mempelajari jenis linked list lainnya seperti *Double Linked List* dan *Circular Linked List*.
4. Disarankan untuk mengombinasikan pemahaman teori dengan praktik secara langsung agar konsep struktur data dapat dikuasai dengan baik.

DAFTAR PUSTAKA

Arna, Fariza Yuliana Setiowati. *Praktikum 15–16: Single Linked List*. Politeknik Elektronika Negeri Surabaya.Tersedia pada: Microsoft Word - Praktikum 15-16 - SingleLinkedList

Teknohole. (2025, Maret 12). *Struktur data single linked list*. Diakses dari https://teknohole.com/article/read/struktur-data-single-linked-list-2025-03-12-073056